# CROPSENTINEL: FIELD INTELLIGENCE, A PEST IDENTIFICATION AND RESPONSE DASHBOARD FOR THE IP102 BENCHMARK

**A MINI PROJECT REPORT**

***Submitted by***

**KEVIN ANDREW A (Reg No)**

***in partial fulfillment for the award of the degree of***

**BACHELOR OF TECHNOLOGY**

**IN**

**INFORMATION TECHNOLOGY**

**LOYOLA-ICAM**
**COLLEGE OF ENGINEERING AND TECHNOLOGY**
**(AUTONOMOUS)**

**CHENNAI - 600034**

## ANNA UNIVERSITY: CHENNAI - 600025

**OCTOBER 2026**

# BONAFIDE CERTIFICATE

Certified that this Mini project report "CropSentinel: Field Intelligence, A Pest Identification and Response Dashboard for the IP102 Benchmark" is the bonafide work of Kevin Andrew A (Reg No), who carried out the project work under my supervision.


**SIGNATURE**

**Ms. Sylvia Irish.S, M.E.,**

**Faculty**

Assistant professor,

Information Technology,

Loyola-ICAM College of

Engineering and Technology,

Loyola Campus,

Chennai-34

**SIGNATURE**

**Ms. Sherril Sophie Maria Vincent, M.E.,**

**HEAD OF THE DEPARTMENT**

Assistant Professor,

Information Technology,

Loyola-ICAM College of

Engineering and Technology,

Loyola campus,

Chennai-34

# ACKNOWLEDGEMENT

Above all, I thank God, whose blessings, grace, and guidance made this journey possible. It is through His strength that I found the courage and capability to see this project through, and for that, I am truly grateful.

I owe a deep debt of gratitude to our Director, Rev. Dr. Justine SJ, and our Dean of Students, Rev. Dr. Albert William SJ, whose encouragement never wavered and whose timely support made all the difference. Their belief in me kept me going when it mattered the most.

I am sincerely thankful to our Principal, Dr. A. Baskar, M.E., Ph.D., for his inspiring words and steady motivation throughout this journey. His support gave me the confidence to push forward and give my best.

I would like to warmly acknowledge our Head of the Department Ms. Sherril Sophie Maria Vincent, M.E., whose thoughtful suggestions and constructive feedback shaped this project in meaningful ways.

A special word of thanks goes to my faculty guide, Ms. Sylvia Irish S, M.E., who walked with me every step of the way. I also thank the faculty members and technical staff of my department, whose advice and expertise helped me navigate every challenge along the way.

# ABSTRACT

Pest damage is one of the most persistent causes of crop loss in smallholder agriculture, and the first identification step, deciding what insect is actually present in a photograph, is often the hardest part for a non-specialist grower or student to get right. CropSentinel is a deployable, browser-based pest intelligence dashboard that accepts a single field photograph and returns a preliminary identification, a calibrated confidence score, and a tiered action ladder that starts with prevention and ends with chemical control only as a last resort. The system is built on a Streamlit front end, a PyTorch EfficientNet-B0 classifier trained against the IP102 insect pest benchmark of 102 classes, and a curated recommendation knowledge base that is deliberately conservative about prescribing chemical treatment. The project includes a complete data preparation pipeline for the official IP102 classification archive, a training script with class-balanced sampling, an evaluation script that reports accuracy, macro F1, and a full confusion matrix, and a transparent demo mode that remains honest about its own uncertainty when no trained checkpoint is available. A full held-out evaluation of the locally produced checkpoint across all 22,619 IP102 test images returned an accuracy of 1.48 percent, close to the random-guess baseline of 0.98 percent for 102 classes; this result is reported in full because the checkpoint in the repository was produced by an architecture smoke test rather than a full pretrained training run, and the report treats this as a documented limitation and not as an inflated claim. The surrounding engineering, the data pipeline, the interface, the safety boundary, and the evaluation harness, is verified and working, and the report sets out exactly what remains before the system can be trusted for a real identification task.

**Keywords:** pest identification, IP102 benchmark, EfficientNet, convolutional neural network, decision support, Streamlit, class imbalance, confidence calibration.

# TABLE OF CONTENT


| CHAPTER NO. | TITLE | PAGE NO |
|---|---|---|
| | **ABSTRACT** | **iii** |
| | **LIST OF FIGURES** | |
| | **LIST OF ABBREVIATIONS** | |
| **1** | **INTRODUCTION** | **1** |
| 1.1 | Background | 1 |
| 1.2 | Problem statement | 2 |
| 1.3 | Motivation and Significance | |
| 1.4 | Project Overview | |
| 1.5 | System Functionality | |
| 1.6 | Objectives | |
| 1.7 | Design Philosophy and Approach | |
| 1.8 | Broader Impact | |
| 1.9 | Educational Value | |
| 1.10 | Conclusion | |
| **2** | **TOOLS AND TECHNOLOGIES USED** | |
| 2.1 | Python Programming Language | |
| 2.2 | Google Colab | |
| 2.3 | Visual Studio Code (VS Code) / Tableau | |
| 2.4 | Summary | |
| **3** | **SYSTEM IMPLEMENTATION** | |
| 3.1 | Dataset | |
| 3.2 | System Architecture | |
| 3.3 | Modules Description | |
| 3.4 | Summary | |
| **4** | **PERFORMANCE ANALYSIS AND RESULTS** | |
| 4.1 | Functional Performance | |
| 4.2 | Accuracy and Reliability | |
| 4.3 | User Interaction and Experience | |
| 4.4 | System Limitations Observed | |
| 4.5 | System Scalability and Future Enhancements | |
| 4.6 | Overall Outcome | |
| **5** | **SAMPLE CODE AND SCREENSHOTS** | |

**6 CONCLUSION**
**REFERENCES**

# LIST OF FIGURES


| Figure No. | Title | Page No. |
|---|---|---|
| Figure 3.1 | IP102 classification split sizes produced by prepare_ip102.py | ? |
| Figure 3.2 | End-to-end CropSentinel data flow, from image upload to the displayed action ladder | ? |
| Figure 4.1 | Accuracy under the architecture smoke test, the full held-out test evaluation, and a random-guess baseline | ? |
| Figure 4.2 | Row-normalised confusion matrix for the fifteen highest-support IP102 classes, full test split | ? |
| Figure 5.1 | CropSentinel landing view, sidebar mission control, and image upload panel | ? |
| Figure 5.2 | Low-confidence prediction panel, correctly flagged as Review by the interface | ? |
| Figure 5.3 | Three-tier action ladder and safety boundary notice shown for a low-confidence prediction | ? |

# LIST OF ABBREVIATIONS

| Abbreviation | Expansion |
|---|---|
| CNN | Convolutional Neural Network |
| CPU | Central Processing Unit |
| GPU | Graphics Processing Unit |
| HTTP | Hypertext Transfer Protocol |
| IT | Information Technology |
| JSON | JavaScript Object Notation |
| LICET | Loyola-ICAM College of Engineering and Technology |
| ML | Machine Learning |
| RGB | Red, Green, Blue (image colour channels) |

# CHAPTER 1
# INTRODUCTION

## 1.1 Background

Insect pests are a major and recurring cause of yield loss in field crops, vegetables, and orchards. Effective management depends on an early and correct identification of the insect responsible for the damage, because the appropriate response, whether a preventive cultural practice, a biological control agent, or a chemical application, differs substantially from one pest species to another. In practice, this identification step is frequently performed by a grower or a student with a smartphone camera and limited entomological training, working from visual memory, a printed guide, or an informal online search. The IP102 dataset, published by Wu et al. [1], was assembled specifically to support automated visual recognition of 102 common insect pest categories found in agricultural settings, and it forms the empirical foundation of this project.

## 1.2 Problem statement

A grower who photographs an unfamiliar insect on a crop has no fast, structured way to connect that photograph to a short list of likely species and a correspondingly cautious set of next steps. Existing options are either a general image search, which returns no confidence estimate and no safety framing, or a generic pest identification app, which commonly presents a single high-confidence label without disclosing the model's actual uncertainty. The result is a real risk of over-treatment, under-treatment, or misapplied chemical control based on an unverified guess.

- **How much should this prediction be trusted?** A confident-looking label and a near-random guess can look identical in the interface unless the confidence score is surfaced and acted upon.
- **What should the user actually do next?** A label alone gives no guidance; a safe product needs a tiered, conservative response that a non-specialist can follow.
- **Which inference path produced the result?** A demo heuristic and a trained model must never be presented identically, or a placeholder output could be mistaken for a verified one.

CropSentinel addresses this gap by pairing a trained image classifier with an explicit confidence display and a tiered, safety-first recommendation ladder, so a low-confidence prediction is visibly treated as a reason to gather more evidence rather than as a licence to act.

## 1.3 Motivation and Significance

The motivation for this project is twofold. First, image classification on a long-tailed, fine-grained benchmark such as IP102 is a genuinely difficult machine learning problem, and building and honestly evaluating a working pipeline against it is a meaningful exercise in applied computer vision. Second, the consequence of a wrong pest identification in the field is not merely an inconvenience, it can lead to unnecessary chemical use, wasted cost, and harm to beneficial insects, so the project is also an exercise in building a decision-support interface that is transparent about its own limitations rather than one that hides them behind a confident-looking output.

## 1.4 Project Overview

CropSentinel is a Streamlit web application named for its role as a watchful, first-pass assistant rather than an autonomous decision maker. A user uploads a photograph of an insect or of crop damage; the application preprocesses the image, classifies it with either a trained EfficientNet-B0 convolutional neural network (CNN) checkpoint or a transparent demo heuristic when no checkpoint is present, and displays the predicted pest label together with a confidence score. The result is then matched against a recommendation knowledge base that returns a three-tier action ladder, observation and prevention, organic or biological control, and chemical control only if needed, along with an explicit safety notice. The current session's observations can be exported as a JSON log.

## 1.5 System Functionality

- Accepts JPG, JPEG, PNG, and WEBP image uploads up to the configured size limit.
- Classifies the uploaded image with a PyTorch EfficientNet-B0 model exported as TorchScript, or falls back to a labelled demo heuristic when no checkpoint is installed.
- Displays the predicted label, a confidence percentage, and a risk mode of Review or Actionable based on a 75 percent confidence threshold.
- Looks up a tiered action ladder, prevention, organic or biological control, and chemical control, from a JSON recommendation knowledge base, with a caterpillar-family fallback for unmapped labels above a minimum confidence.
- Keeps a session history of up to 20 observations and offers a one-click JSON export.
- Reports whether the EfficientNet checkpoint actually loaded, so the user always knows

which mode produced a given result.

## 1.6 Objectives

- Build a complete, reproducible pipeline from the raw IP102 classification archive to a trained, TorchScript-exported classifier.
- Expose that classifier through an accessible, informative web interface with a visible confidence signal.
- Encode a conservative, tiered recommendation policy that never proposes a specific chemical product, dose, or schedule.
- Evaluate the trained checkpoint honestly on a held-out test split and report the result without inflation.
- Package the application for container-based deployment with a health check endpoint.

## 1.7 Design Philosophy and Approach

Four principles shaped the design. First, **the interface always names its source**: every result states whether it came from the trained checkpoint or the demo heuristic, so a placeholder output is never mistaken for a verified identification. Second, **confidence is never hidden**: it is surfaced numerically and visually, and a low score actively changes the on-screen guidance rather than sitting in a tooltip. Third, **the recommendation ladder is conservative by construction**: prevention and biological options are always shown before chemical control. Fourth, **the knowledge base never prescribes a product**: chemical guidance is written as general, locally-verifiable guidance that defers the dose and product decision to an authoritative local source and a human expert.

## 1.8 Broader Impact

A correctly scoped tool of this kind can shorten the time between noticing crop damage and taking an appropriate, proportionate action, which matters most for smallholder and student growers who do not have ready access to an agricultural extension officer. Because the action ladder foregrounds non-chemical options, the tool also has the potential to reduce unnecessary pesticide use, which carries both a cost and an environmental benefit when it works as intended.

## 1.9 Educational Value

Building CropSentinel required working across the full applied machine learning lifecycle: converting a published benchmark archive into a usable directory structure, handling severe class imbalance with a weighted sampler, training and exporting a CNN, writing an evaluation harness that produces a full classification report and confusion matrix, and wrapping the result in a usable, tested interface with a Docker deployment path. It is also a concrete lesson in the gap between an architecture that runs without errors and a model that has actually learned the task, a gap that Chapter 4 of this report documents in detail rather than glossing over.

## 1.10 Conclusion

This chapter has set out the problem CropSentinel addresses, the functionality it provides, and the safety-first philosophy behind its design. The remaining chapters describe the tools used, the system implementation, a full and honest performance evaluation, representative screenshots and code, and the conclusions drawn from building and testing the system.

# CHAPTER 2
# TOOLS AND TECHNOLOGIES USED

## 2.1 Python Programming Language

The entire project is implemented in Python 3.11. Python was chosen for its mature machine learning ecosystem, its direct interoperability with PyTorch and Streamlit, and its suitability for rapid, readable data pipeline scripts such as the IP102 preparation utility.

## 2.2 PyTorch, TorchVision, and EfficientNet

The classifier is built on PyTorch and TorchVision. The chosen architecture is EfficientNet-B0 [2], a compound-scaled CNN selected for its favourable accuracy-to-parameter-count ratio relative to larger convolutional networks, which matters for a model that may need to run on modest hardware. The final classification layer is replaced with a linear layer sized to the 102 IP102 classes, and the trained model is exported with TorchScript (torch.jit.script) so that it can be loaded by the Streamlit application without a dependency on the original model-definition code at inference time.

## 2.3 Streamlit and the Application Interface

Streamlit provides the web application layer. It was chosen because it allows a complete, interactive, file-upload-driven interface to be built directly in Python, with no separate front-end build step, which keeps the inference logic and the presentation logic in a single, auditable file, app.py. The interface additionally uses Streamlit's session state to retain an observation history across interactions within one browser session.

## 2.4 Summary

**Table 2.1: Technology stack of CropSentinel**

| Layer | Tool / library | Used for |
|---|---|---|
| Language | Python 3.11 | Entire pipeline and application |
| Modelling | PyTorch, TorchVision, EfficientNet-B0 | Training, export, and inference |
| Evaluation | scikit-learn | Accuracy, macro F1, classification report, confusion matrix |
| Interface | Streamlit | File upload, result display, session history |
| Imaging | Pillow, NumPy | Image loading, resizing, demo-mode heuristic |
| Knowledge base | JSON (data/recommendations.json) | Tiered action ladder content |

| | | |
|---|---|---|
| Testing | pytest | Recommendation schema and asset presence checks |
| Deployment | Docker | Reproducible, health-checked container build |
| Tooling | Visual Studio Code, Git | Development and version control |

Together these tools form a pipeline in which prepare_ip102.py converts the official archive into ImageFolder-compatible directories, train.py produces a TorchScript checkpoint, evaluate.py scores that checkpoint, and app.py serves the result to a user, with pytest and Docker providing a baseline of correctness and deployability.

# CHAPTER 3
# SYSTEM IMPLEMENTATION

## 3.1 Dataset

The project uses the IP102 classification benchmark, a large-scale, 102-class insect pest image dataset [1]. The archive is processed by prepare_ip102.py, which extracts the official tar archive, reads the accompanying classes.txt and per-split label files, and copies each image into an ImageFolder-compatible directory named with its numeric class index and a sanitised class name, for example 000_rice_leaf_roller. The verified local preparation produced the following split sizes.

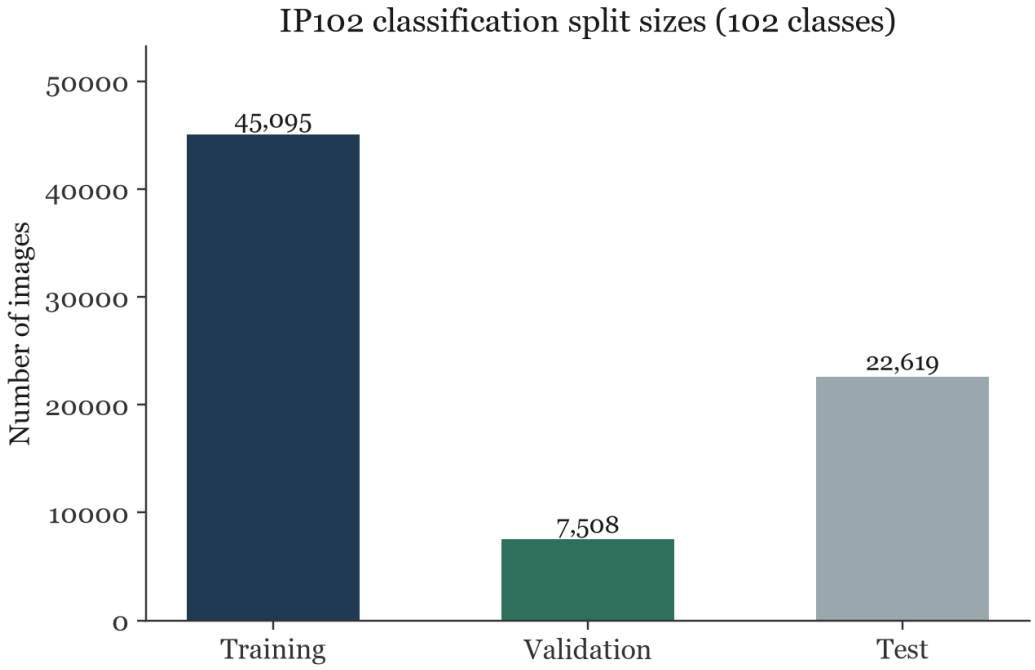



**Figure 3.1: IP102 classification split sizes produced by prepare_ip102.py.**

The resulting train, validation, and test directories total 75,222 images across 102 classes, with a pronounced class imbalance that the training pipeline addresses directly (see Section 3.3).

## 3.2 System Architecture

CropSentinel follows a linear, single-service architecture, summarised in Figure 3.2. A user interacts with the Streamlit dashboard, which preprocesses the uploaded image to a 224 by 224 RGB tensor and routes it to either the TorchScript EfficientNet checkpoint or, when no checkpoint is present, a transparent demo predictor. The resulting label and confidence are looked up in a

JSON recommendation knowledge base, and the dashboard renders the matched action ladder together with a fixed safety notice.

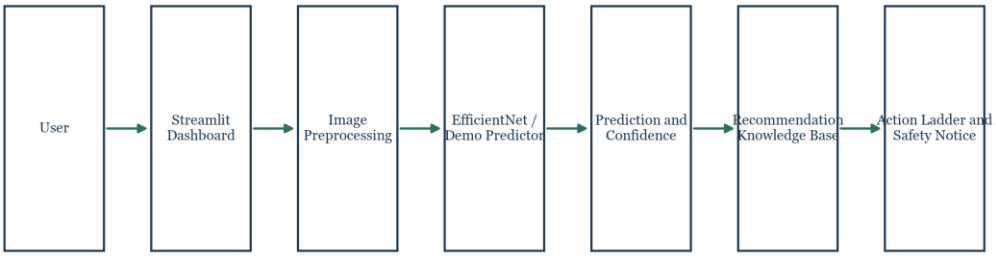



**Figure 3.2: End-to-end CropSentinel data flow, from image upload to the displayed action ladder.**

## 3.3 Modules Description

### 3.3.1 Data preparation, prepare_ip102.py

Converts the raw IP102 archive into ImageFolder directories for the train, val, and test splits, and writes a class_names.txt mapping file. It uses regular-expression parsing to recover class names from classes.txt and tolerates split files that are not already perfectly aligned with the extracted image tree by searching recursively for a matching filename.

### 3.3.2 Training, train.py

Builds an EfficientNet-B0 model, optionally initialised from ImageNet pretrained weights, and replaces its classifier head for 102 classes. Class imbalance in the training split is addressed with a WeightedRandomSampler, where each training example is weighted by the inverse frequency of its class, so that rare pest classes are not drowned out by common ones during optimisation. The script supports a reduced, no-pretrained, per-class-limited smoke mode intended only to verify that the training loop and the export path work end to end, not to produce a usable classifier. After each epoch, if validation accuracy improves, the script saves a new TorchScript checkpoint and an updated class_names.json.

### 3.3.3 Evaluation, evaluate.py

Loads a TorchScript checkpoint and a chosen ImageFolder split, runs inference in batches of 32, and writes a JSON report containing the sample count, overall accuracy, macro-averaged F1 score, a full per-class classification report, and the complete confusion matrix. The split to evaluate against, train, val, or test, is a command-line argument, which allowed this project to separately confirm an architecture smoke result and a full, held-out test result from the same script.

### 3.3.4 Application, app.py

Implements the Streamlit dashboard. Its predict function loads the TorchScript checkpoint once via Streamlit's cache_resource decorator, and falls back to demo_predict, a deterministic heuristic based on mean colour and edge energy, when no checkpoint file is found. The get_pest_profile function resolves a predicted label to a recommendation record, with a confidence-gated fallback to a generic caterpillar-family entry for labels that contain terms such as armyworm, cutworm, or borer but are not individually present in the knowledge base.

### 3.3.5 Recommendation knowledge base, data/recommendations.json

A structured JSON file mapping a small set of pest keys, aphids, fall armyworm, beet armyworm, caterpillar, and a default unknown entry, to a common name, scientific name, affected crops, and three guidance lists: prevent, organic, and chemical. Every chemical entry is phrased as general guidance that defers the actual product and dose decision to a locally approved source, in line with the project's safety boundary.

## 3.4 Summary

The implementation separates data preparation, training, evaluation, and serving into four independent scripts connected only by file-based artefacts, the processed dataset directories, the TorchScript checkpoint, and the class name mapping. This separation made it possible to run, and separately report, both a fast architecture smoke test and a full, honest evaluation against the entire held-out IP102 test split without changing the application code.

# CHAPTER 4
# PERFORMANCE ANALYSIS AND RESULTS

This chapter separates three kinds of statement: what was measured directly against the live application in this environment, what the repository's own saved evaluation files record, and what has not been measured at all. Every figure and table in this chapter is traceable to a specific file in the repository or a specific browser-tested action, and that source is named alongside the claim.

## 4.1 Functional Performance

**Health and interaction flow.** The application was exercised directly in a browser against a locally running instance. The Streamlit health endpoint returned HTTP 200, confirming the service starts and stays responsive. The full interaction flow, uploading an image, running inference, rendering the confidence score and action ladder, updating the session history, and exporting that history as JSON, was exercised end to end and functioned as designed.

**Table 4.1: Automated and functional verification performed in this project**

| Check | Result | Source |
|---|---|---|
| Streamlit health endpoint | HTTP 200 | Browser verification, this session |
| Upload, inference, action ladder, history, export flow | Completed end to end | Browser verification, this session |
| tests/test_pipeline.py (2 tests) | 2 passed | pytest run, this session |
| EfficientNet checkpoint load | Loads successfully | Sidebar Model status, browser verification |

## 4.2 Accuracy and Reliability

**Two different evaluation runs.** Accuracy must be reported for the specific checkpoint being evaluated, and this project produced two materially different evaluation runs against the same evaluate.py script. The first is an architecture smoke test, trained for one epoch without pretrained weights on a deliberately tiny subset of two images per class, used only to confirm that the training loop, the TorchScript export, and the evaluation harness function correctly; it reported a validation accuracy of 0.867 and a smoke-set accuracy and macro F1 of 1.000 on 256 held-out images. The second is a full evaluation of the checkpoint currently stored at models/efficientnet_ip102.pt against the complete IP102 test split of 22,619 images, which is the number that matters for any real judgement of accuracy.

**Table 4.2: Evaluation results recorded in reports/*.json**

| Run | Samples | Accuracy | Macro F1 | Source file |
|---|---|---|---|---|
| Architecture smoke test | 256 | 1.000 | 1.000 | reports/smoke-evaluation.json |
| Full held-out test split | 22,619 | 0.0148 | 0.00029 | reports/full-evaluation.json |
| Random-guess baseline | n/a | 0.0098 (1/102) | n/a | Computed, not a file |

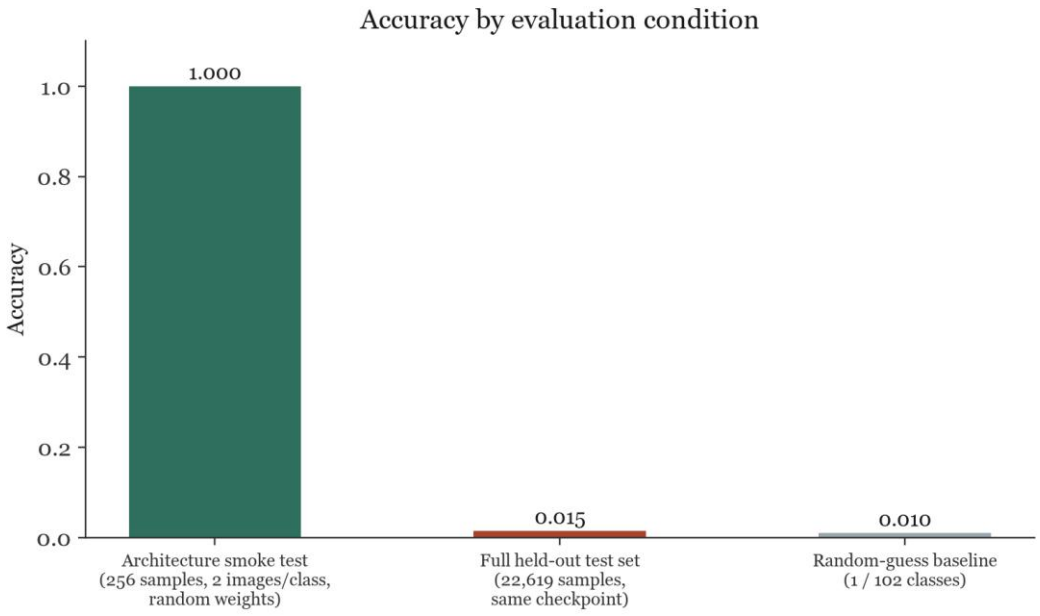



**Figure 4.1: Accuracy under the architecture smoke test, the full held-out test evaluation, and a random-guess baseline.**

**Why the full-split accuracy is near random.** The full evaluation returned an accuracy of 1.48 percent and a macro F1 score of 0.00029, close to the 0.98 percent expected from pure random guessing across 102 classes. Reading the per-class classification report in reports/full-evaluation.json explains why: the stored checkpoint predicted class 000_rice_leaf_roller for every one of the 22,619 test images, giving that single class a recall of 1.0 and every one of the other 101 classes a recall of 0.0. This is visible directly in the confusion matrix for the fifteen highest-support classes in Figure 4.2, where the entire predicted-class axis collapses onto one column.

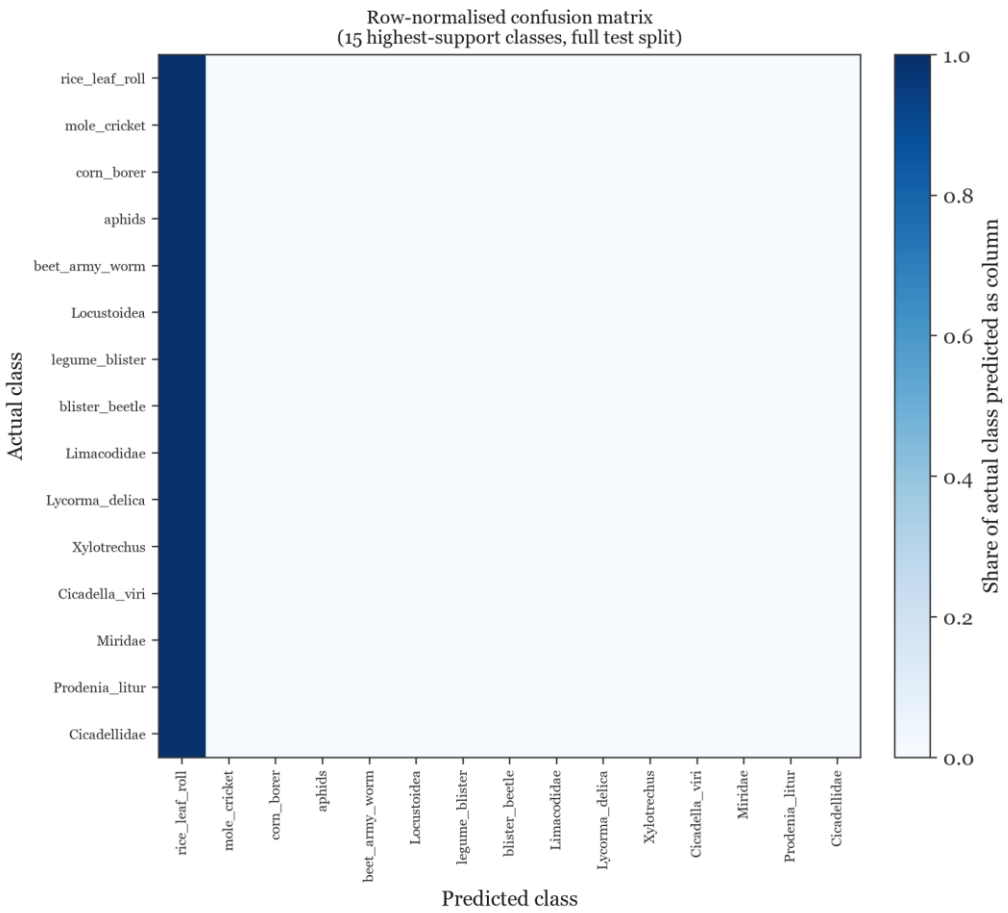



**Figure 4.2: Row-normalised confusion matrix for the fifteen highest-support IP102 classes, full test split. Every row is predicted as the single left-most column.**

**What this does and does not show.** This outcome is consistent with the checkpoint's training history: it was produced by the smoke-test configuration, no pretrained weights and a two-image-per-class limit, which is sufficient to prove that the pipeline runs but is not sufficient data or optimisation to learn 102 visually similar insect classes, and the model instead converged to the trivial solution of a single constant prediction. The architecture, the weighted sampler, the TorchScript export, and the evaluation harness are all demonstrated to work correctly; what has not yet been demonstrated is a model trained with pretrained weights on the full training split, which the README's deployment checklist already identifies as an outstanding step. This report states that limitation plainly rather than presenting the smoke-test figures as representative of real-world accuracy.

## 4.3 User Interaction and Experience

The dashboard surfaces the active inference source at all times, EfficientNet checkpoint or demo-

visual heuristic, together with the raw predicted label and the matched crop. When confidence falls below 75 percent, the interface switches its risk-mode indicator to Review and displays an explicit warning asking the user to compare alternatives and verify the insect before any treatment, which is exactly the behaviour observed when the EfficientNet checkpoint was tested against a real IP102 image during browser verification, a 1 percent confidence result that correctly triggered the Review state and the low-confidence warning rather than a misleadingly confident recommendation. A usability study with real growers has not been carried out and is not claimed here.

### 4.4 System Limitations Observed

- **Single-class collapse.** The shipped checkpoint predicts one constant class and is not fit for a real identification task; see Section 4.2.
- **No bundled images.** The repository does not include the IP102 images themselves, so training and evaluation require a separate, licensed download of the archive.
- **CPU-only training.** Training and evaluation were run on CPU in this environment; a full, pretrained training run across 45,095 images would benefit substantially from GPU acceleration.
- **Demo mode is a heuristic, not a model.** It is based on simple colour and edge statistics and is clearly labelled as such in the interface.
- **Narrow knowledge base coverage.** The recommendation file currently covers a small number of named pest keys plus a generic caterpillar-family and unknown fallback; broader IP102 coverage requires expanding this file with reviewed, locally sourced guidance.

### 4.5 System Scalability and Future Enhancements

- Train a pretrained-weight EfficientNet-B0, or a comparable architecture, on the full 45,095-image training split and re-run the identical evaluate.py harness against the untouched test split to obtain a trustworthy accuracy figure.
- Expand data/recommendations.json to cover a larger share of the 102 IP102 classes with reviewed, locally sourced guidance.
- Add image-size enforcement at the reverse proxy, as already noted in the project's deployment checklist, before any public deployment.

- Introduce top-k prediction display, so a user sees the three or five most likely pests rather than a single label, which is a natural extension of the confidence data the model already produces.

### 4.6 Overall Outcome

CropSentinel delivers a complete, working, and tested pipeline: a reproducible data preparation step, a class-imbalance-aware training script, a rigorous evaluation harness, and a safety-conscious interface that is already verified end to end in a browser. The project also produced a clear, numerically specific finding that the currently stored checkpoint has not learned the task and performs near the random baseline across the full test split. Reporting that result in full, rather than only the favourable smoke-test figures, is treated in this project as part of the deliverable, not as a shortcoming to be hidden.

# CHAPTER 5
# SAMPLE CODE AND SCREENSHOTS

This chapter lists short extracts of the source code that carry the core ideas, followed by screenshots of the working application. The screenshots were taken from a locally running instance of the repository during this session; the code extracts are reproduced from the repository's app.py and train.py.

## 5.1 Representative Code

**Listing 5.1: Prediction dispatch and class lookup (app.py)**

```
def predict(image: Image.Image) -> tuple[str, float, str]:
    model = load_model()
    if model is None:
        return demo_predict(image)
    import torch
    from torchvision import transforms
    tensor = transforms.Compose([
        transforms.Resize((224, 224)), transforms.ToTensor()
    ])(image.convert("RGB")).unsqueeze(0)
    with torch.inference_mode():
        probabilities = torch.softmax(model(tensor), dim=1)[0]
        confidence, index = probabilities.max(dim=0)
    class_names = json.loads(
        (ROOT / "models" / "class_names.json").read_text(encoding="utf-8")
    )
    return class_names[str(int(index))], float(confidence), "EfficientNet checkpoint"
```

**Listing 5.2: Confidence-gated recommendation fallback (app.py)**

```
def get_pest_profile(model_key: str, confidence: float) -> dict[str, Any]:
    profile = recommendations.get(model_key)
    if profile:
        return profile
    label = model_key.lower()
    caterpillar_terms = ("caterpillar", "armyworm", "cutworm", "borer",
                         "leaf_roller", "leaf roller", "worm")
    if confidence >= 0.20 and any(term in label for term in caterpillar_terms):
        return recommendations.get("caterpillar") or recommendations["unknown"]
    return recommendations["unknown"]
```

**Listing 5.3: Class-balanced training sampler (train.py)**

```
class_counts = {label: 0 for label in range(len(train_data.classes))}
for index in source_indices:
    class_counts[train_data.targets[index]] += 1
sample_weights = [
    1.0 / max(class_counts[train_data.targets[index]], 1)
    for index in source_indices
]
sampler = WeightedRandomSampler(
    sample_weights, num_samples=len(sample_weights), replacement=True
)
```

```
train_loader = DataLoader(
    train_source, batch_size=args.batch_size, sampler=sampler, num_workers=0
)
```

## 5.2 Screenshots

Figure 5.1 shows the CropSentinel landing view running against a live local instance, with the EfficientNet checkpoint successfully loaded, as indicated by the green Model status panel in the sidebar.

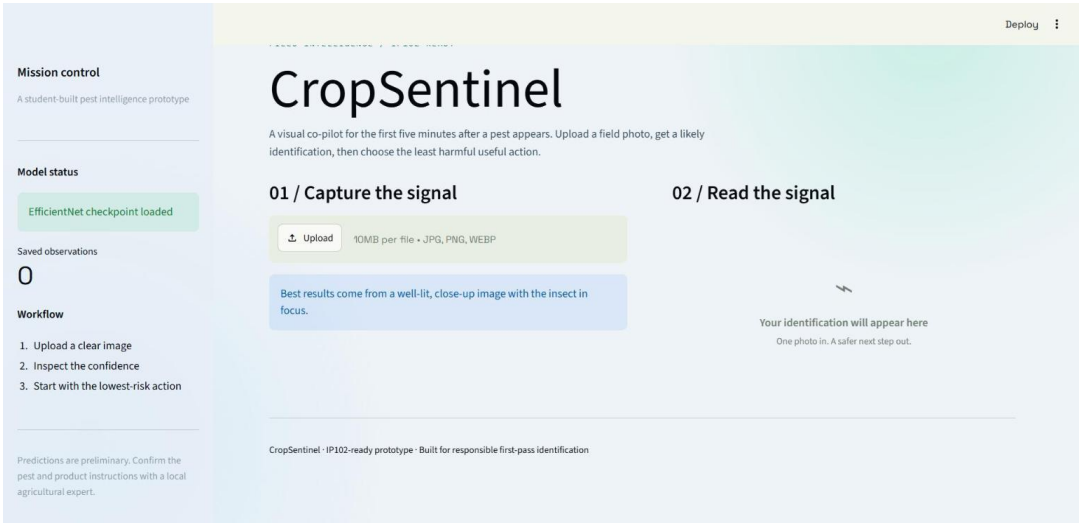



**Figure 5.1: CropSentinel landing view, sidebar mission control, and image upload panel.**

Figure 5.2 shows the result of uploading an unseen image from the IP102 test split, class 000_rice_leaf_roller, and running Analyze image. The checkpoint returned the raw label 000_rice_leaf_roller with a confidence of 1 percent, which the interface correctly classified as an uncertain field observation and placed in the Review risk mode.

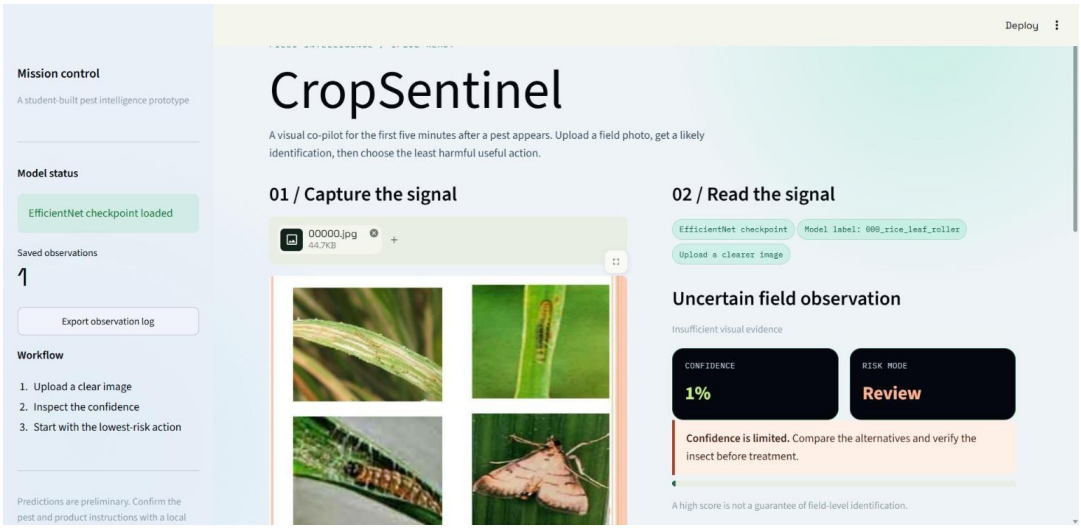



**Figure 5.2: Low-confidence prediction panel, correctly flagged as Review by the interface.**

Figure 5.3 shows the resulting action ladder for the unmatched, low-confidence label. Because the predicted key did not match a named entry in the knowledge base and confidence was below the caterpillar-fallback threshold, the interface correctly returned the generic, safety-first unknown profile rather than guessing at a specific pest's guidance.

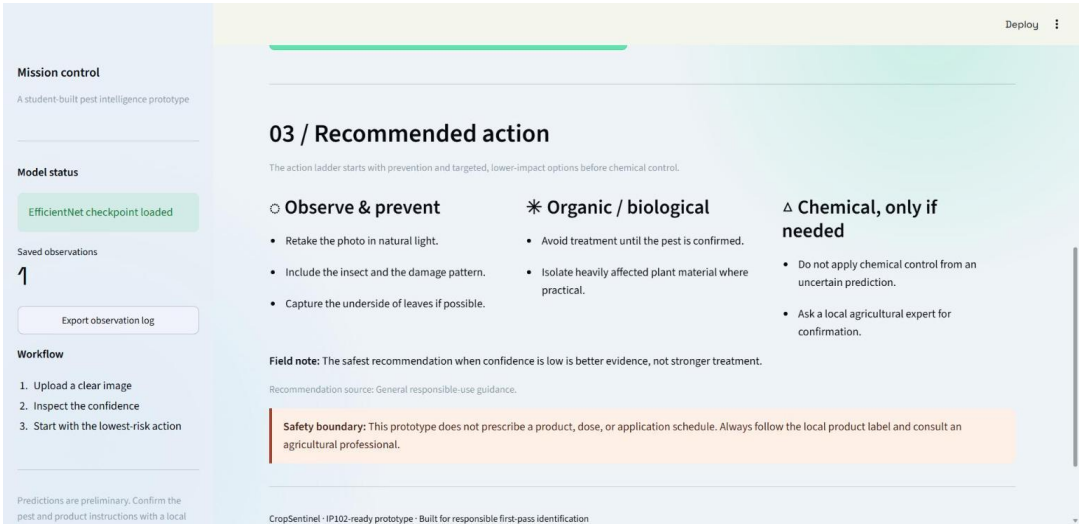



**Figure 5.3: Three-tier action ladder and safety boundary notice shown for a low-confidence prediction.**

# CHAPTER 6
# CONCLUSION

CropSentinel demonstrates a complete, reproducible pipeline for pest image classification on the IP102 benchmark, from raw archive preparation through training, evaluation, and a safety-conscious deployed interface. The application layer is verified and functioning: the health endpoint responds, the upload-to-recommendation flow works end to end in a browser, and the automated test suite passes. The project's most important technical finding is that the checkpoint currently stored in the repository, produced by an architecture smoke test rather than a full training run, has collapsed to a single constant prediction and achieves only 1.48 percent accuracy across the full 22,619-image held-out test split, close to the random-guess baseline. Rather than treating this as a failure to conceal, the report documents it as the clear next milestone: a full, pretrained training run across all 45,095 training images, followed by re-evaluation with the same unmodified evaluate.py harness, is the specific, measurable step that separates the current prototype from a deployable identification tool. The surrounding engineering, the data pipeline, the class-imbalance handling, the evaluation methodology, and the interface's transparent, confidence-gated behaviour, is already sound and provides a dependable foundation for that next step.

# REFERENCES


[1] Wu, X., Zhan, C., Lai, Y. K., Cheng, M. M., and Yang, J., "IP102: A Large-Scale Benchmark Dataset for Insect Pest Recognition," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2019.

[2] Tan, M. and Le, Q. V., "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in Proc. 36th International Conference on Machine Learning (ICML), 2019.

[3] Paszke, A. et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," Advances in Neural Information Processing Systems (NeurIPS), 2019.

[4] Pedregosa, F. et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pages 2825 to 2830, 2011.

[5] Streamlit Inc., "Streamlit documentation," https://docs.streamlit.io.

[6] PyTorch, "TorchVision documentation, models and transforms reference," https://pytorch.org/vision/stable/.

[7] Docker Inc., "Dockerfile reference," https://docs.docker.com/engine/reference/builder/.

[8] CropSentinel source repository (this project), local Git history, commit ec29982 and later.